Pipelined MIPS Processor with Hazard Detection Simulated in QuestaSim

NAME		SRN	
NAME		SRN	
NAME		SRN	
NAME		SRN	

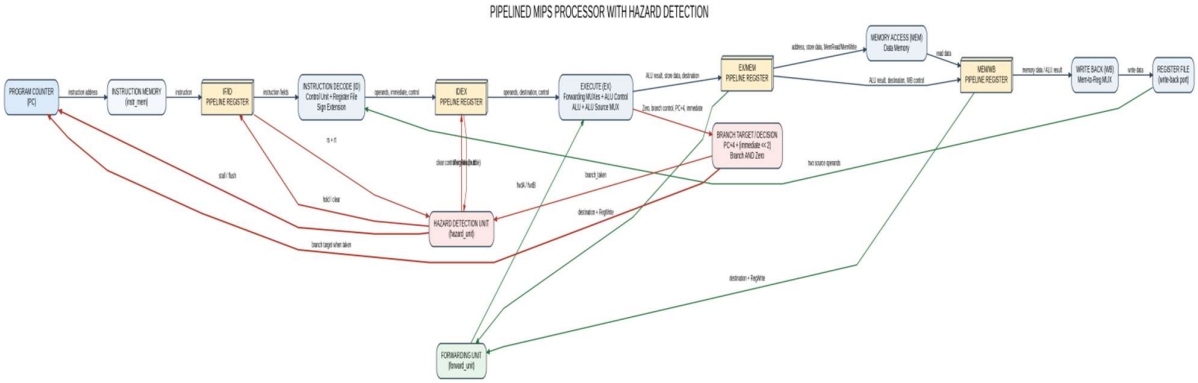
1. PROBLEM STATEMENT

The objective of this project is to design and simulate a **Pipelined MIPS Processor with Hazard Detection** using Verilog HDL in QuestaSim. The processor implements instruction execution through a five-stage pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).

Pipelining improves processor performance by allowing multiple instructions to be processed simultaneously at different stages of execution. However, it introduces potential hazards such as data hazards and control hazards, which may affect the correct execution of instructions.

The project focuses on implementing pipeline registers, hazard detection, data forwarding, and pipeline flushing mechanisms to ensure correct instruction execution and improve overall processor efficiency.

2. CIRCUIT DIAGRAM



The proposed design consists of a five-stage pipelined MIPS processor. Each stage performs a specific operation, and pipeline registers are used to transfer intermediate data and control signals between consecutive stages.

The five stages of the processor are:

1. **Instruction Fetch (IF):** Fetches instructions from instruction memory using the Program Counter (PC).
2. **Instruction Decode (ID):** Decodes the instruction, generates control signals, and reads operands from the register file.
3. **Execute (EX):** Performs arithmetic and logical operations using the Arithmetic Logic Unit (ALU).
4. **Memory Access (MEM):** Performs read and write operations on data memory.
5. **Write Back (WB):** Writes the final result into the destination register.

The stages are connected using four pipeline registers: IF/ID, ID/EX, EX/MEM, and MEM/WB.

The processor also consists of a Control Unit, Register File, ALU Control Unit, Forwarding Unit, Hazard Detection Unit, Instruction Memory, and Data Memory.

Working of the Circuit

The processor fetches instructions sequentially using the Program Counter. The fetched instructions are decoded, and the required operands are obtained from the Register File. The ALU performs the required operations, and the Data Memory handles load and store instructions. Finally, the results are written back into the destination registers.

The Forwarding Unit resolves data dependencies by selecting the required result from the EX/MEM or MEM/WB pipeline registers. The Hazard Detection Unit identifies load-use hazards and generates stall signals when necessary. The processor also supports branch flushing to remove instructions that should not be executed after a branch is taken.

3. TRUTH TABLE

3.1 Main Control Unit Truth Table

The Main Control Unit generates control signals according to the opcode of the instruction.

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type	1	0	0	1	0	0	0	10
LW	0	1	1	1	1	0	0	00
SW	0	1	0	0	0	1	0	00
BEQ	0	0	0	0	0	0	1	01
ADDI	0	1	0	1	0	0	0	00

Description of Control Signals:

- RegDst: Selects the destination register.
- ALUSrc: Selects the second ALU operand.
- MemtoReg: Selects the data to be written back.
- RegWrite: Enables register writing.
- MemRead: Enables memory reading.
- MemWrite: Enables memory writing.
- Branch: Indicates a branch instruction.
- ALUOp: Determines the ALU operation.

3.2 ALU Control Unit Truth Table

The ALU Control Unit determines the operation to be performed by the ALU using the ALUOp and function fields.

ALUOp	Function Code	ALU Control	Operation
00	X	0010	ADD
01	X	0110	SUB
10	100000	0010	ADD
10	100010	0110	SUB
10	100100	0000	AND
10	100101	0001	OR
10	101010	0111	SLT

3.3 ALU Truth Table

ALU Control	Operation	Function
0010	ADD	Performs addition
0110	SUB	Performs subtraction
0000	AND	Performs bitwise AND
0001	OR	Performs bitwise OR
0111	SLT	Sets output to 1 if A is less than B

The ALU also generates a Zero signal, which becomes 1 when the ALU result is zero.

3.4 Forwarding Unit Truth Table

The Forwarding Unit identifies data dependencies and selects the appropriate operand for the ALU.

Condition	Forward Signal	Selected Data
No dependency	00	ID/EX register value
EX/MEM dependency	10	EX/MEM ALU result
MEM/WB dependency	01	MEM/WB write-back result

The forwarding signals are generated independently for ALU inputs A and B. When both pipeline stages contain matching destination registers, EX/MEM forwarding is given priority.

3.5 Hazard Detection Unit Truth Table

The Hazard Detection Unit detects load-use hazards by comparing the destination register of a load instruction with the source register fields of the following instruction.

The stall condition is:

```
stall = ID_EX_memread AND
        (ID_EX_rt != 0) AND
        ((ID_EX_rt == IF_ID_rs) OR
         (ID_EX_rt == IF_ID_rt))
```

The flush condition is:

```
flush = branch_taken
```

Condition	Stall	Flush
Normal execution	0	0
Load-use hazard detected	1	0
Branch taken	0	1

When a stall occurs, the PC and IF/ID register are held, and the ID/EX control signals are cleared to insert a bubble.

When a branch is taken, the processor updates the PC with the branch target and clears the relevant pipeline registers to remove incorrect instructions.

4. FINITE STATE MACHINE

The Finite State Machine (FSM) is not applicable to this project because the processor control logic is implemented using combinational logic rather than a sequential state machine. Instruction execution follows a fixed five-stage pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). These stages represent the sequence of instruction execution and do not constitute an FSM.

5. BRIEF LITERATURE SURVEY

5.1 Computer Organization and Design – David A. Patterson and John L. Hennessy

The book *Computer Organization and Design: The Hardware/Software Interface* by David A. Patterson and John L. Hennessy provides a detailed explanation of processor architecture, instruction execution, datapath design, and pipelining. It explains how processor performance can be improved by dividing instruction execution into multiple stages. It also discusses data hazards, forwarding, pipeline stalls, and control hazards, which form the theoretical foundation of this project.

5.2 Pipelined Processor Architecture

Pipelining is an important technique used in computer architecture to improve instruction throughput. It divides instruction execution into multiple stages so that different instructions can be processed simultaneously. The five-stage MIPS architecture provides a structured approach to understanding instruction execution and pipeline operation. This concept is used in the proposed processor to improve hardware resource utilization.

5.3 Data Forwarding and Hazard Detection

Data forwarding and hazard detection are essential techniques used to maintain correct instruction execution in pipelined processors. Forwarding allows the processor to obtain results directly from later pipeline stages without waiting for register write-back. Hazard detection identifies instruction dependencies that cannot be resolved immediately and introduces stalls when necessary.

5.4 Pipeline Stalling and Flushing

Pipeline stalling is used when an instruction must wait for the availability of required data. A bubble is inserted into the pipeline to prevent incorrect execution. Pipeline flushing is used during branch operations to remove instructions fetched from an incorrect execution path. These techniques help maintain the correctness of the processor.

6. VERILOG CODE

6.1) alu.v

```
module alu(
    input  [31:0] a, b,
    input  [3:0]  ctrl,
    output reg [31:0] y,
    output        zero
);
    always @(*) begin
        case (ctrl)
            4'b0010: y = a + b;
            4'b0110: y = a - b;
            4'b0000: y = a & b;
            4'b0001: y = a | b;
            4'b0111: y = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0;
            default: y = 32'b0;
        endcase
    end

    assign zero = (y == 32'b0);
endmodule
```

6.2)alu_control.v

```
module alu_control(
    input  [1:0] aluop,
    input  [5:0] funct,
    output reg [3:0] ctrl
);
    always @(*) begin
        case (aluop)
            2'b00: ctrl = 4'b0010;
            2'b01: ctrl = 4'b0110;
            2'b10: begin
                case (funct)
                    6'b100000: ctrl = 4'b0010;
                    6'b100010: ctrl = 4'b0110;
                    6'b100100: ctrl = 4'b0000;
                    6'b100101: ctrl = 4'b0001;
                    6'b101010: ctrl = 4'b0111;
                    default:   ctrl = 4'b0010;
                endcase
            end
            default: ctrl = 4'b0010;
        endcase
    end
endmodule
```

6.3)control_unit.v

```
module control_unit(
    input  [5:0] opcode,
    output reg regdst, alusrc, memtoreg, regwrite,
    output reg memread, memwrite, branch,
    output reg [1:0] aluop
);
    always @(*) begin
        regdst = 0; alusrc = 0; memtoreg = 0; regwrite = 0;
        memread = 0; memwrite = 0; branch = 0;
        aluop = 2'b00;

        case (opcode)
            6'b000000: begin regdst = 1; regwrite = 1; aluop = 2'b10; end
            6'b100011: begin alusrc = 1; memtoreg = 1; regwrite = 1; memread = 1; end
            6'b101011: begin alusrc = 1; memwrite = 1; end
            6'b000100: begin branch = 1; aluop = 2'b01; end
            6'b001000: begin alusrc = 1; regwrite = 1; end
default: ;
        endcase
    end
endmodule
```

6.4)data_mem.v

```
module data_mem(
    input         clk,
    input         memread, memwrite,
    input  [31:0] addr, wd,
    output [31:0] rd
);
    reg [31:0] mem [0:63];
    integer i;

    initial begin
        for (i = 0; i < 64; i = i + 1) mem[i] = 32'b0;
    end

    assign rd = memread ? mem[addr[7:2]] : 32'b0;

    always @(posedge clk) begin
        if (memwrite)
            mem[addr[7:2]] <= wd;
    end
endmodule
```

6.5)forward_unit.v

```
module forward_unit(
    input  [4:0] ID_EX_rs, ID_EX_rt,
    input  [4:0] EX_MEM_rd, MEM_WB_rd,
    input        EX_MEM_regwrite, MEM_WB_regwrite,
    output reg [1:0] fwdA, fwdB
);
    always @(*) begin
        fwdA = 2'b00;
        fwdB = 2'b00;

        if (EX_MEM_regwrite && (EX_MEM_rd != 5'd0) && (EX_MEM_rd == ID_EX_rs))
            fwdA = 2'b10;
        else if (MEM_WB_regwrite && (MEM_WB_rd != 5'd0) && (MEM_WB_rd == ID_EX_rs))
            fwdA = 2'b01;

        if (EX_MEM_regwrite && (EX_MEM_rd != 5'd0) && (EX_MEM_rd == ID_EX_rt))
            fwdB = 2'b10;
        else if (MEM_WB_regwrite && (MEM_WB_rd != 5'd0) && (MEM_WB_rd == ID_EX_rt))
            fwdB = 2'b01;
    end
endmodule
```

6.6)hazard_unit.v

```
module hazard_unit(
    input        ID_EX_memread,
    input  [4:0] ID_EX_rt, IF_ID_rs, IF_ID_rt,
    input        branch_taken,
    output       stall,
    output       flush
);
    assign stall = ID_EX_memread && (ID_EX_rt != 5'd0) &&
                   ((ID_EX_rt == IF_ID_rs) || (ID_EX_rt == IF_ID_rt));

    assign flush = branch_taken;
endmodule
```

6.7)instr_mem.v

```
module instr_mem(
    input  [31:0] addr,
    output [31:0] instr
);
    reg [31:0] mem [0:63];
    integer i;

    initial begin
        for (i = 0; i < 64; i = i + 1) mem[i] = 32'b0;

        mem[0]  = 32'h2001000A;
        mem[1]  = 32'h20020014;
        mem[2]  = 32'h00221820;
        mem[3]  = 32'h00612022;
        mem[4]  = 32'h00622824;
        mem[5]  = 32'h00613025;
        mem[6]  = 32'hAC040000;
        mem[7]  = 32'h8C070000;
        mem[8]  = 32'h00E14020;
        mem[9]  = 32'h0022482A;
        mem[10] = 32'h10210002;
        mem[11] = 32'h200A0063;
        mem[12] = 32'h200B0063;
        mem[13] = 32'h200C0005;
        mem[14] = 32'h10220001;
        mem[15] = 32'h200D0007;
        mem[16] = 32'h018D7020;
        mem[17] = 32'h1000FFFF;
    end

    assign instr = mem[addr[7:2]];
endmodule
```

6.8)mips_pipeline.v

```
module mips_pipeline(
    input clk,
    input reset
);
    reg [31:0] pc;

    reg [31:0] IF_ID_instr, IF_ID_pc4;

    reg        ID_EX_regdst, ID_EX_alusrc, ID_EX_memtoreg, ID_EX_regwrite;
    reg        ID_EX_memread, ID_EX_memwrite, ID_EX_branch;
    reg [1:0]  ID_EX_aluop;
    reg [31:0] ID_EX_pc4, ID_EX_rd1, ID_EX_rd2, ID_EX_imm;
    reg [4:0]  ID_EX_rs, ID_EX_rt, ID_EX_rd;

    reg        EX_MEM_memtoreg, EX_MEM_regwrite, EX_MEM_memread, EX_MEM_memwrite;
    reg [31:0] EX_MEM_alu, EX_MEM_store;
    reg [4:0]  EX_MEM_rd;

    reg        MEM_WB_memtoreg, MEM_WB_regwrite;
    reg [31:0] MEM_WB_memdata, MEM_WB_alu;
    reg [4:0]  MEM_WB_rd;

    wire        stall, flush;
    wire        branch_taken;
    wire [31:0] branch_target;
    wire [31:0] wb_data;

    // ================ IF stage ================
    wire [31:0] instr;
    wire [31:0] pc_plus4 = pc + 32'd4;

    instr_mem im(.addr(pc), .instr(instr));

    always @(posedge clk) begin
        if (reset)            pc <= 32'b0;
        else if (flush)       pc <= branch_target;
        else if (!stall)      pc <= pc_plus4;
    end
```

```
    // ---------- IF/ID ----------
    always @(posedge clk) begin
        if (reset || flush) begin
            IF_ID_instr <= 32'b0;
            IF_ID_pc4   <= 32'b0;
        end else if (!stall) begin
            IF_ID_instr <= instr;
            IF_ID_pc4   <= pc_plus4;
        end
    end

    // ================ ID stage ================
    wire [5:0]  opcode = IF_ID_instr[31:26];
    wire [4:0]  rs     = IF_ID_instr[25:21];
    wire [4:0]  rt     = IF_ID_instr[20:16];
    wire [4:0]  rd     = IF_ID_instr[15:11];

    wire [31:0] rd1, rd2, imm_ext;
    wire        c_regdst, c_alusrc, c_memtoreg, c_regwrite;
    wire        c_memread, c_memwrite, c_branch;
    wire [1:0]  c_aluop;

    control_unit cu(
        .opcode(opcode),
        .regdst(c_regdst), .alusrc(c_alusrc), .memtoreg(c_memtoreg),
        .regwrite(c_regwrite), .memread(c_memread), .memwrite(c_memwrite),
        .branch(c_branch), .aluop(c_aluop)
    );

    reg_file rf(
        .clk(clk), .we(MEM_WB_regwrite),
        .ra1(rs), .ra2(rt),
        .wa(MEM_WB_rd), .wd(wb_data),
        .rd1(rd1), .rd2(rd2)
    );

    sign_extend se(.a(IF_ID_instr[15:0]), .y(imm_ext));
```

```
hazard_unit hu(
    .ID_EX_memread(ID_EX_memread), .ID_EX_rt(ID_EX_rt),
    .IF_ID_rs(rs), .IF_ID_rt(rt),
    .branch_taken(branch_taken),
    .stall(stall), .flush(flush)
);

// ---------- ID/EX ----------
always @(posedge clk) begin
    if (reset || stall || flush) begin
        ID_EX_regdst <= 0;  ID_EX_alusrc <= 0;  ID_EX_memtoreg <= 0; ID_EX_regwrite <= 0;
        ID_EX_memread <= 0; ID_EX_memwrite <= 0; ID_EX_branch <= 0;  ID_EX_aluop <= 2'b00;
        ID_EX_pc4 <= 0; ID_EX_rd1 <= 0; ID_EX_rd2 <= 0; ID_EX_imm <= 0;
        ID_EX_rs <= 0;  ID_EX_rt <= 0;  ID_EX_rd <= 0;
    end else begin
        ID_EX_regdst <= c_regdst;     ID_EX_alusrc <= c_alusrc;
        ID_EX_memtoreg <= c_memtoreg; ID_EX_regwrite <= c_regwrite;
        ID_EX_memread <= c_memread;   ID_EX_memwrite <= c_memwrite;
        ID_EX_branch <= c_branch;     ID_EX_aluop <= c_aluop;
        ID_EX_pc4 <= IF_ID_pc4;       ID_EX_rd1 <= rd1;
        ID_EX_rd2 <= rd2;             ID_EX_imm <= imm_ext;
        ID_EX_rs <= rs;               ID_EX_rt <= rt;   ID_EX_rd <= rd;
    end
end

// ================ EX stage ================
wire [1:0] fwdA, fwdB;

forward_unit fu(
    .ID_EX_rs(ID_EX_rs), .ID_EX_rt(ID_EX_rt),
    .EX_MEM_rd(EX_MEM_rd), .MEM_WB_rd(MEM_WB_rd),
    .EX_MEM_regwrite(EX_MEM_regwrite), .MEM_WB_regwrite(MEM_WB_regwrite),
    .fwdA(fwdA), .fwdB(fwdB)
);

reg [31:0] alu_in_a, alu_in_b_reg;
```

```
    always @(*) begin
        case (fwdA)
            2'b10:   alu_in_a = EX_MEM_alu;
            2'b01:   alu_in_a = wb_data;
            default: alu_in_a = ID_EX_rd1;
        endcase

        case (fwdB)
            2'b10:   alu_in_b_reg = EX_MEM_alu;
            2'b01:   alu_in_b_reg = wb_data;
            default: alu_in_b_reg = ID_EX_rd2;
        endcase
    end

    wire [31:0] alu_in_b = ID_EX_alusrc ? ID_EX_imm : alu_in_b_reg;
    wire [3:0]  alu_ctrl;
    wire [31:0] alu_y;
    wire        zero;

    alu_control ac(.aluop(ID_EX_aluop), .funct(ID_EX_imm[5:0]), .ctrl(alu_ctrl));
    alu         u_alu(.a(alu_in_a), .b(alu_in_b), .ctrl(alu_ctrl), .y(alu_y), .zero(zero));

    wire [4:0] ex_dest = ID_EX_regdst ? ID_EX_rd : ID_EX_rt;

    assign branch_target = ID_EX_pc4 + (ID_EX_imm << 2);
    assign branch_taken  = ID_EX_branch & zero;

    // ---------- EX/MEM ----------
    always @(posedge clk) begin
        if (reset) begin
            EX_MEM_memtoreg <= 0; EX_MEM_regwrite <= 0;
            EX_MEM_memread  <= 0; EX_MEM_memwrite <= 0;
            EX_MEM_alu <= 0; EX_MEM_store <= 0; EX_MEM_rd <= 0;
        end else begin
            EX_MEM_memtoreg <= ID_EX_memtoreg; EX_MEM_regwrite <= ID_EX_regwrite;
            EX_MEM_memread  <= ID_EX_memread;  EX_MEM_memwrite <= ID_EX_memwrite;
            EX_MEM_alu      <= alu_y;
            EX_MEM_store    <= alu_in_b_reg;
            EX_MEM_rd       <= ex_dest;
        end
    end
```

```
    // ================= MEM stage =================
    wire [31:0] mem_rd;

    data_mem dm(
        .clk(clk),
        .memread(EX_MEM_memread), .memwrite(EX_MEM_memwrite),
        .addr(EX_MEM_alu), .wd(EX_MEM_store),
        .rd(mem_rd)
    );

    // ---------- MEM/WB ----------
    always @(posedge clk) begin
        if (reset) begin
            MEM_WB_memtoreg <= 0; MEM_WB_regwrite <= 0;
            MEM_WB_memdata <= 0;  MEM_WB_alu <= 0; MEM_WB_rd <= 0;
        end else begin
            MEM_WB_memtoreg <= EX_MEM_memtoreg; MEM_WB_regwrite <= EX_MEM_regwrite;
            MEM_WB_memdata  <= mem_rd;
            MEM_WB_alu      <= EX_MEM_alu;
            MEM_WB_rd       <= EX_MEM_rd;
        end
    end

    // ================= WB stage =================
    assign wb_data = MEM_WB_memtoreg ? MEM_WB_memdata : MEM_WB_alu;

endmodule
```

6.9)reg_file.v

```
    // ================= MEM stage =================
    wire [31:0] mem_rd;

    data_mem dm(
        .clk(clk),
        .memread(EX_MEM_memread), .memwrite(EX_MEM_memwrite),
        .addr(EX_MEM_alu), .wd(EX_MEM_store),
        .rd(mem_rd)
    );

    // ---------- MEM/WB ----------
    always @(posedge clk) begin
        if (reset) begin
            MEM_WB_memtoreg <= 0; MEM_WB_regwrite <= 0;
            MEM_WB_memdata <= 0;  MEM_WB_alu <= 0; MEM_WB_rd <= 0;
        end else begin
            MEM_WB_memtoreg <= EX_MEM_memtoreg; MEM_WB_regwrite <= EX_MEM_regwrite;
            MEM_WB_memdata  <= mem_rd;
            MEM_WB_alu      <= EX_MEM_alu;
            MEM_WB_rd       <= EX_MEM_rd;
        end
    end

    // ================= WB stage =================
    assign wb_data = MEM_WB_memtoreg ? MEM_WB_memdata : MEM_WB_alu;

endmodule
```

7. TESTBENCH CODE

7.1) tb_mips.v

```
`timescale 1ns/1ps

module tb_mips;
    reg clk, reset;
    integer i;

    mips_pipeline dut(.clk(clk), .reset(reset));

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_mips);

        reset = 1;
        #12;
        reset = 0;

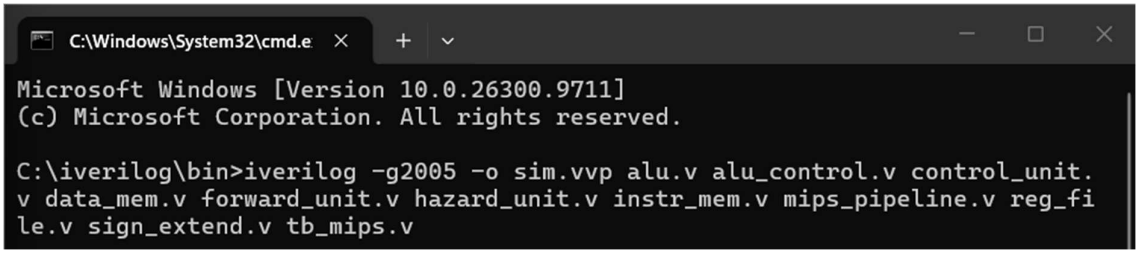
        #600;

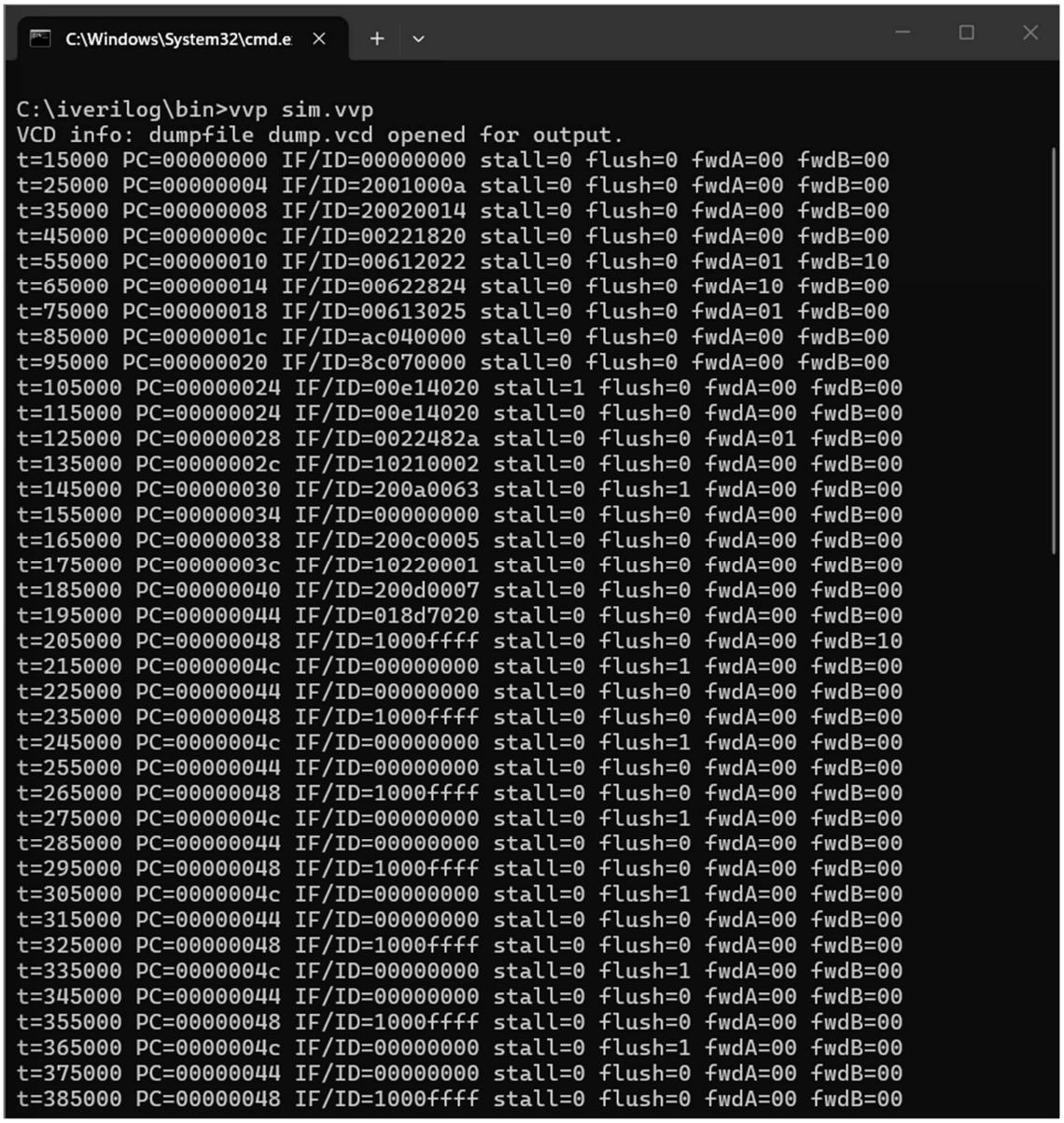
        $display("========== FINAL REGISTER VALUES ==========");
        for (i = 0; i < 16; i = i + 1)
            $display("$%0d = %0d", i, dut.rf.regs[i]);
        $display("mem[0] = %0d", dut.dm.mem[0]);
        $display("==========================================");
        $finish;
    end

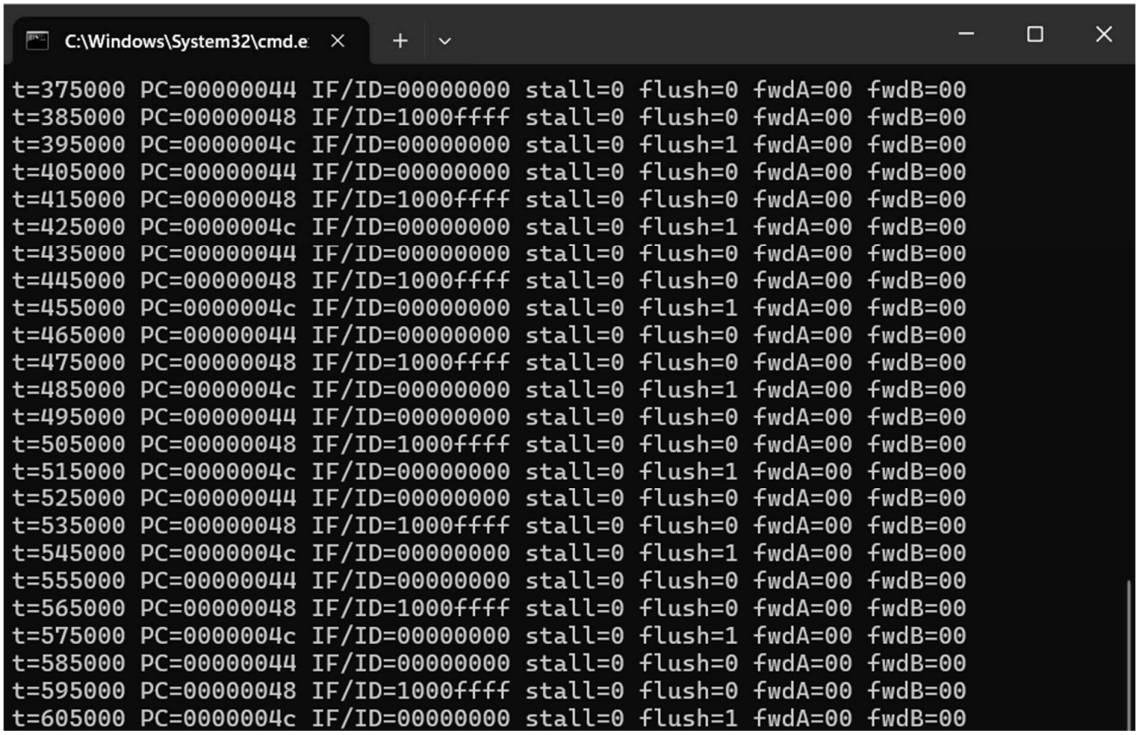
    always @(posedge clk) begin
        if (!reset)
            $display("t=%0t PC=%h IF/ID=%h stall=%b flush=%b fwdA=%b fwdB=%b",
                     $time, dut.pc, dut.IF_ID_instr,
                     dut.stall, dut.flush, dut.fwdA, dut.fwdB);
    end
endmodule
```

8. OUTPUT

8.1) TERMINAL OUTPUT

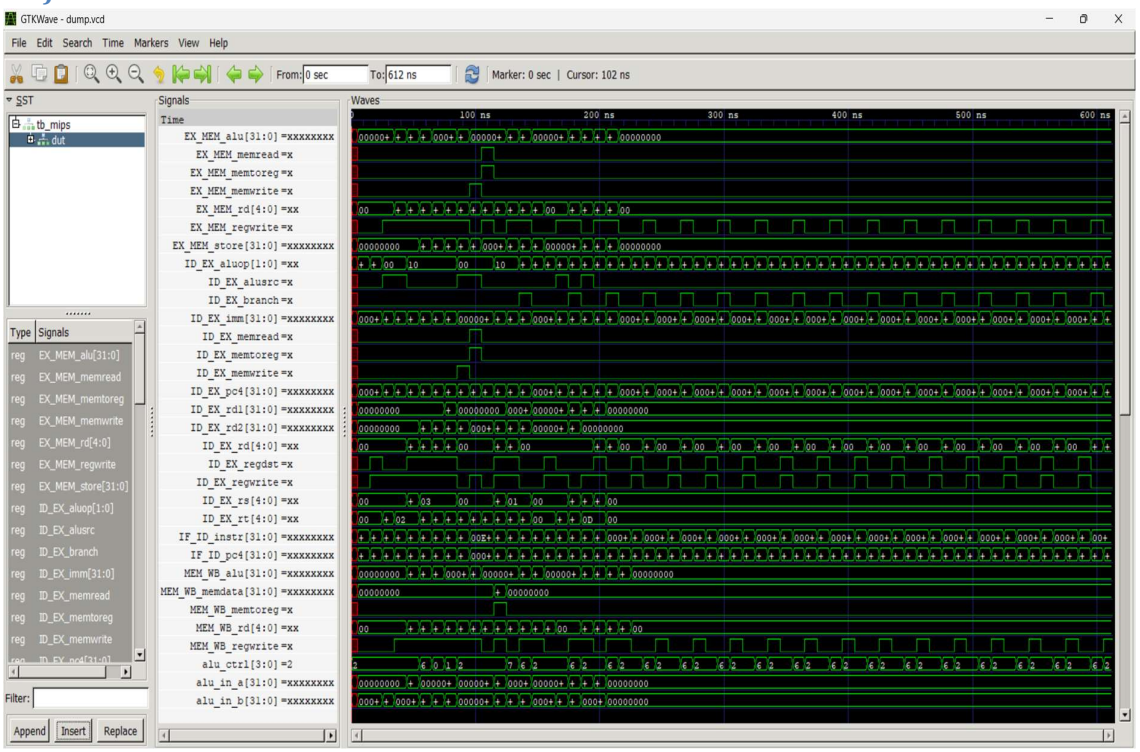


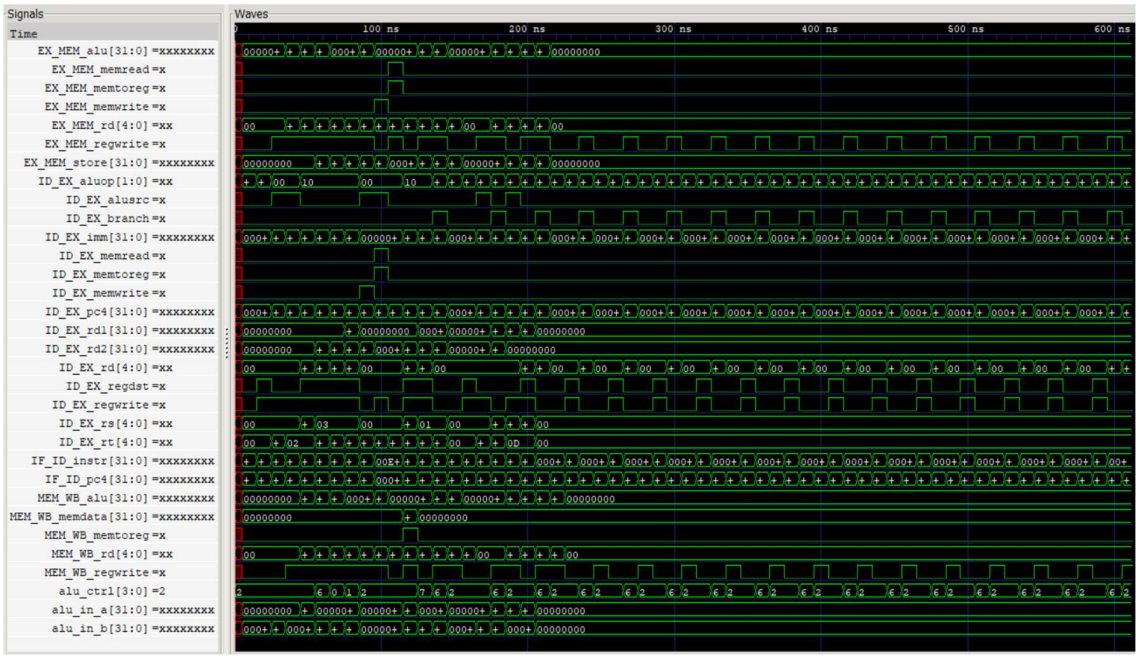


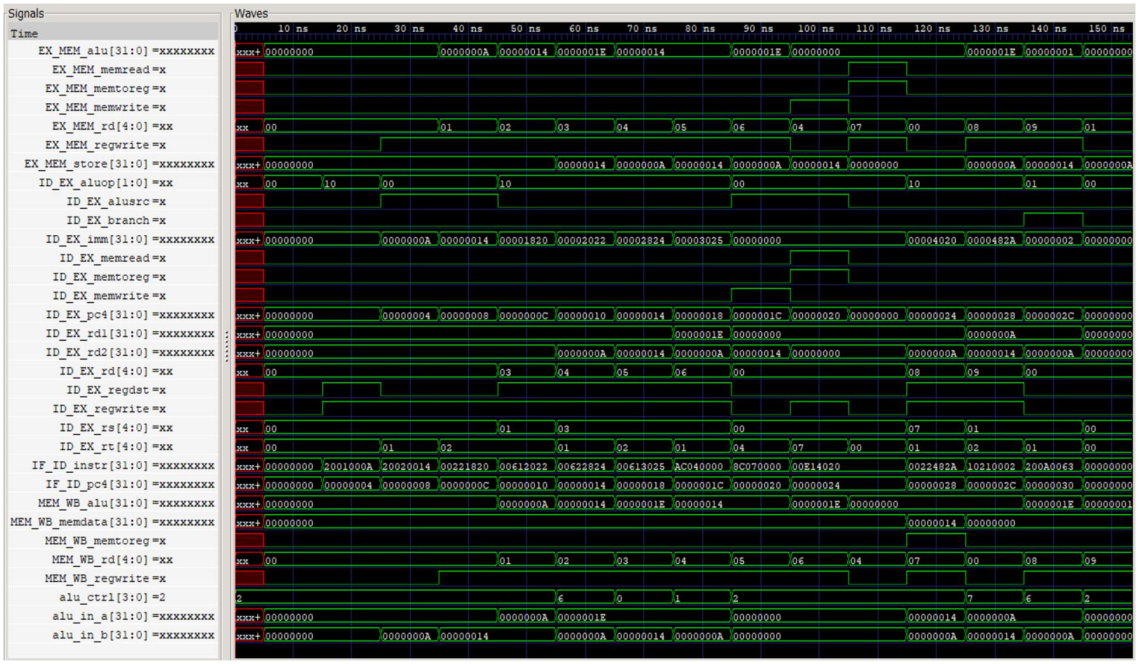

C:\Windows\System32\cmd.e
t=375000 PC=00000044 IF/ID=00000000 stall=0 flush=0 fwdA=00 fwdB=00
t=385000 PC=00000048 IF/ID=1000ffff stall=0 flush=0 fwdA=00 fwdB=00
t=395000 PC=0000004c IF/ID=00000000 stall=0 flush=1 fwdA=00 fwdB=00
t=405000 PC=00000044 IF/ID=00000000 stall=0 flush=0 fwdA=00 fwdB=00
t=415000 PC=00000048 IF/ID=1000ffff stall=0 flush=0 fwdA=00 fwdB=00
t=425000 PC=0000004c IF/ID=00000000 stall=0 flush=1 fwdA=00 fwdB=00
t=435000 PC=00000044 IF/ID=00000000 stall=0 flush=0 fwdA=00 fwdB=00
t=445000 PC=00000048 IF/ID=1000ffff stall=0 flush=0 fwdA=00 fwdB=00
t=455000 PC=0000004c IF/ID=00000000 stall=0 flush=1 fwdA=00 fwdB=00
t=465000 PC=00000044 IF/ID=00000000 stall=0 flush=0 fwdA=00 fwdB=00
t=475000 PC=00000048 IF/ID=1000ffff stall=0 flush=0 fwdA=00 fwdB=00
t=485000 PC=0000004c IF/ID=00000000 stall=0 flush=1 fwdA=00 fwdB=00
t=495000 PC=00000044 IF/ID=00000000 stall=0 flush=0 fwdA=00 fwdB=00
t=505000 PC=00000048 IF/ID=1000ffff stall=0 flush=0 fwdA=00 fwdB=00
t=515000 PC=0000004c IF/ID=00000000 stall=0 flush=1 fwdA=00 fwdB=00
t=525000 PC=00000044 IF/ID=00000000 stall=0 flush=0 fwdA=00 fwdB=00
t=535000 PC=00000048 IF/ID=1000ffff stall=0 flush=0 fwdA=00 fwdB=00
t=545000 PC=0000004c IF/ID=00000000 stall=0 flush=1 fwdA=00 fwdB=00
t=555000 PC=00000044 IF/ID=00000000 stall=0 flush=0 fwdA=00 fwdB=00
t=565000 PC=00000048 IF/ID=1000ffff stall=0 flush=0 fwdA=00 fwdB=00
t=575000 PC=0000004c IF/ID=00000000 stall=0 flush=1 fwdA=00 fwdB=00
t=585000 PC=00000044 IF/ID=00000000 stall=0 flush=0 fwdA=00 fwdB=00
t=595000 PC=00000048 IF/ID=1000ffff stall=0 flush=0 fwdA=00 fwdB=00
t=605000 PC=0000004c IF/ID=00000000 stall=0 flush=1 fwdA=00 fwdB=00


========== FINAL REGISTER VALUES ==========
$0 = 0
$1 = 10
$2 = 20
$3 = 30
$4 = 20
$5 = 20
$6 = 30
$7 = 20
$8 = 30
$9 = 1
$10 = 0
$11 = 0
$12 = 5
$13 = 7
$14 = 12
$15 = 0
mem[0] = 20
==========
C:\iverilog\bin>

8.2) GTK WAVEFORM







9. CONCLUSION

The project successfully demonstrates the design and simulation of a five-stage Pipelined MIPS Processor with Hazard Detection using Verilog HDL. The processor consists of five stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB), connected through pipeline registers.

The implementation of the Forwarding Unit helps resolve data dependencies between instructions, while the Hazard Detection Unit identifies load-use hazards and generates appropriate stall signals. Pipeline flushing is also implemented to handle branch instructions and prevent incorrect instruction execution.

The processor is verified using a Verilog test bench, and its functionality is analyzed through simulation outputs and waveforms. The project provides practical understanding of pipelined processor architecture, data forwarding, hazard detection, pipeline stalling, and flushing mechanisms.

Overall, this project enhances the understanding of computer architecture and digital system design while demonstrating how pipelining improves instruction throughput and how hazard-handling techniques maintain correct processor operation.